

S-Expressions + Cons

CS-3180

Lisp Functions

- We talked about how functions are called
- Using `prefix` notation
- `(fn args...)`
- Consider this lisp code:

```
(max 1 6)
```

S-Expressions

- All of Lisp code is made up of two parts
- The "Literals"
- The "Lists"
- Call these two pieces together: S-Expressions

```
(max 1 6)
```

Atoms

- Pieces of Lisp Code that evaluate to themselves
- `5` -> 5
- `"hello"` -> "hello"
- `'FOO` -> FOO
- Even if FOO is a function, without a call, it evaluates to itself
- You can test this in `sbc l`

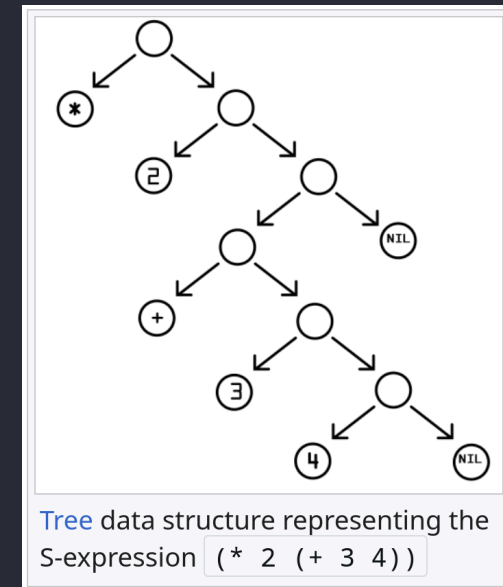
Lists

- Defined via parentheses
- Consists of Atoms OR other Lists
- Outer S-Expression contains:
 - * -> Atom
 - 2 -> Atom
 - (. . .) -> List

```
( * 2 ( + 3 4 ) )
```

S-Expressions

- List is a series of Atoms or Lists
- S-Expression is a series of atoms or list
- An S-Expression is series of Atoms and S-Expressions
- Defined *recursively*



List Execution

- All the Lisp interpreter does is execute lists
- The first value in a list is the function
- Everything else is an argument
- You CANNOT just put parentheses wherever
- Could attempt to call a function called 1

```
(1)
```

Single Quote

- Evaluates to the `quote` function
- Tells the interpreter to "stop interpreting"
- Treat the expression as data, instead of code
- Often useful for literals
- Note: these are not "string" literals

```
(A B C) ; -> Call A(B, C)  
'(A B C) ; -> Returns to me the list containing (A B C)
```

Usefulness

- Benefits of this will come later
- Helpful to know your code is in an easily parsable format
- Much more structured than most languages
- Lists (S-Expressions) are the fundamental data structure of your code
- What other language has code as a DS like this?
- Let's look at how to manipulate lists

Creating Lists

- Construct a node via:
- `(cons val1 val2)`
- A node consists of two pieces
- Returns a cons cell
- Denoted as `(THING1 . THING2)`

```
(cons 'ALICE 'BOB) ; (ALICE . BOB)
(cons 1 2) ; (1 . 2)
```

Linked Lists

- Use cons cells to make singly-linked lists
- Really, all your Lisp IS a singly linked list literal
- (This is part of why Lisp is slow)
- Chain these values together
- We will need to use `nil`

```
1      (cons 'ALICE nil)
2      (cons 'ALICE (cons 'BOB nil))
3      (cons 'ALICE (cons 'BOB (cons 'CHARLIE nil)))
4      ; what do these return?
```

Linked Lists

- Use cons cells to make singly-linked lists
- Really, all your Lisp IS a singly linked list literal
- (This is part of why Lisp is slow)
- Chain these values together
- We will need to use `nil`

```
1      (cons 'ALICE nil)
2      (cons 'ALICE (cons 'BOB nil))
3      (cons 'ALICE (cons 'BOB (cons 'CHARLIE nil)))
4      ; what do these return?
```

Linked Lists

- Use cons cells to make singly-linked lists
- Really, all your Lisp IS a singly linked list literal
- (This is part of why Lisp is slow)
- Chain these values together
- We will need to use `nil`

```
1      (cons 'ALICE nil)
2      (cons 'ALICE (cons 'BOB nil))
3      (cons 'ALICE (cons 'BOB (cons 'CHARLIE nil)))
4      ; what do these return?
```

Linked Lists

- Use cons cells to make singly-linked lists
- Really, all your Lisp IS a singly linked list literal
- (This is part of why Lisp is slow)
- Chain these values together
- We will need to use `nil`

```
1      (cons 'ALICE nil)
2      (cons 'ALICE (cons 'BOB nil))
3      (cons 'ALICE (cons 'BOB (cons 'CHARLIE nil)))
4      ; what do these return?
```

Linked Lists

- Use cons cells to make singly-linked lists
- Really, all your Lisp IS a singly linked list literal
- (This is part of why Lisp is slow)
- Chain these values together
- We will need to use `nil`

```
1      (cons 'ALICE nil)
2      (cons 'ALICE (cons 'BOB nil))
3      (cons 'ALICE (cons 'BOB (cons 'CHARLIE nil)))
4      ; what do these return?
```

Linked Lists

- Use cons cells to make singly-linked lists
- Really, all your Lisp IS a singly linked list literal
- (This is part of why Lisp is slow)
- Chain these values together
- We will need to use `nil`

```
1      (cons 'ALICE nil)
2      (cons 'ALICE (cons 'BOB nil))
3      (cons 'ALICE (cons 'BOB (cons 'CHARLIE nil)))
4      ; what do these return?
```

Graphs

- You are not limited to lists
- Just directed graphs
- Spend time visualizing what these are
- See what they print/evaluate to
- Work backwards

```
(cons (cons 'ALICE 'BOB) (cons 'CHARLIE 'DEREK))
```

Why?

- Why does:
- `(cons (cons 'ALICE 'BOB) (cons 'CHARLIE 'DEREK))`
- Evaluate to:
- `((ALICE . BOB) CHARLIE . DEREK)`
- A singly linked list will print normally
- Expects that notation

Dissection

- Now that we can create them
- Talk about how to pull them apart
- Since each cell is two pieces
- Lisp provides us two functions to pull them apart

CAR Function

- Returns the first element from a list
- Returns this as an element
- Whatever that element (list or not) happens to be
- Consider nested lists

```
1      (car '(A B C))           ; A
2      (car '((A B) C))        ; (A B)
3      (car '((A B C) Z))      ; ?
4      (car (car '((A B C) Z))) ; ?
```

CAR Function

- Returns the first element from a list
- Returns this as an element
- Whatever that element (list or not) happens to be
- Consider nested lists

```
1      (car '(A B C))           ; A
2      (car '((A B) C))         ; (A B)
3      (car '((A B C) Z))       ; ?
4      (car (car '((A B C) Z))) ; ?
```

CAR Function

- Returns the first element from a list
- Returns this as an element
- Whatever that element (list or not) happens to be
- Consider nested lists

```
1      (car '(A B C))           ; A
2      (car '((A B) C))        ; (A B)
3      (car '((A B C) Z))      ; ?
4      (car (car '((A B C) Z))) ; ?
```

CAR Function

- Returns the first element from a list
- Returns this as an element
- Whatever that element (list or not) happens to be
- Consider nested lists

```
1      (car '(A B C))           ; A
2      (car '((A B) C))        ; (A B)
3      (car '((A B C) Z))      ; ?
4      (car (car '((A B C) Z))) ; ?
```

CAR Function

- Returns the first element from a list
- Returns this as an element
- Whatever that element (list or not) happens to be
- Consider nested lists

```
1      (car '(A B C))           ; A
2      (car '((A B) C))        ; (A B)
3      (car '((A B C) Z))      ; ?
4      (car (car '((A B C) Z))) ; ?
```

CAR Function

- Returns the first element from a list
- Returns this as an element
- Whatever that element (list or not) happens to be
- Consider nested lists

```
1      (car '(A B C))           ; A
2      (car '((A B) C))        ; (A B)
3      (car '((A B C) Z))      ; ?
4      (car (car '((A B C) Z))) ; ?
```

CDR Function

- Returns a list containing everything but the first element
- ALWAYS returns as a list
- Even when the "rest" is not

```
1      (CDR '(A B C))          ; (B C)
2      (CDR '(A B))           ; (B)
3      (CDR (cons 'A (cons 'B (cons 'C nil)))) ; ?
4      (CDR (cons 'A (cons 'B (cons 'C 'D))))  ; ?
```

CDR Function

- Returns a list containing everything but the first element
- ALWAYS returns as a list
- Even when the "rest" is not

```
1      (CDR '(A B C))          ; (B C)
2      (CDR '(A B))            ; (B)
3      (CDR (cons 'A (cons 'B (cons 'C nil)))) ; ?
4      (CDR (cons 'A (cons 'B (cons 'C 'D))))  ; ?
```

CDR Function

- Returns a list containing everything but the first element
- ALWAYS returns as a list
- Even when the "rest" is not

```
1      (CDR '(A B C))          ; (B C)
2      (CDR '(A B))           ; (B)
3      (CDR (cons 'A (cons 'B (cons 'C nil)))) ; ?
4      (CDR (cons 'A (cons 'B (cons 'C 'D))))  ; ?
```

CDR Function

- Returns a list containing everything but the first element
- ALWAYS returns as a list
- Even when the "rest" is not

```
1      (CDR '(A B C))          ; (B C)
2      (CDR '(A B))           ; (B)
3      (CDR (cons 'A (cons 'B (cons 'C nil)))) ; ?
4      (CDR (cons 'A (cons 'B (cons 'C 'D))))  ; ?
```

CDR Function

- Returns a list containing everything but the first element
- ALWAYS returns as a list
- Even when the "rest" is not

```
1      (CDR '(A B C))          ; (B C)
2      (CDR '(A B))            ; (B)
3      (CDR (cons 'A (cons 'B (cons 'C nil)))) ; ?
4      (CDR (cons 'A (cons 'B (cons 'C 'D))))  ; ?
```

CDR Function

- Returns a list containing everything but the first element
- ALWAYS returns as a list
- Even when the "rest" is not

```
1      (CDR '(A B C))          ; (B C)
2      (CDR '(A B))           ; (B)
3      (CDR (cons 'A (cons 'B (cons 'C nil)))) ; ?
4      (CDR (cons 'A (cons 'B (cons 'C 'D))))  ; ?
```

Combine CAR, CDR, CONS

- Evaluate some expressions

```
(CDR (CAR (CONS  
          (CONS 'A 'B)  
          (CONS 'C 'D)))) ; ?
```

Combine CAR, CDR, CONS

- Evaluate some expressions

```
(CDR (CAR (CONS  
          (CONS 'A 'B)  
          (CONS 'C 'D)))) ; ?
```

```
(CAR (CDR (CDR  
           (CONS 'A (CONS 'B (CONS 'C nil)))))) ; ?
```

Combine CAR, CDR, CONS

- Evaluate some expressions

```
(CDR (CAR (CONS  
          (CONS 'A 'B)  
          (CONS 'C 'D)))) ; ?
```

```
(CAR (CDR (CDR  
           (CONS 'A (CONS 'B (CONS 'C nil)))))) ; ?
```

- Note: "last" CAR turns (C) into C

LIST Function

- Returns a (singly-linked) list of the elements
- Shorthand for CONS
- Evaluate all of its arguments
- (unlike a quote)

```
(LIST 1 2 3) ; (1 2 3)
(LIST (+ 1 2) (* 2 4) 9) ; (3 8 9)
(LIST (+ 1 1) (* 2 2) 'FOO) ; (2 4 FOO)
```

APPEND Function

- Merges multiple lists into one
- Generally, expects all list elements
- Move the pointers around accordingly
- Careful with this

```
1      (APPEND '(1 2) '(3 4))           ; (1 2 3 4)
2      (APPEND '(1 2) '(3 4) '(5 6))    ; (1 2 3 4 5 6)
3      (APPEND '(A B C) 'D)              ; (A B C . D)
4      (APPEND '(A B C) (LIST 'D))       ; (A B C D)
```

APPEND Function

- Merges multiple lists into one
- Generally, expects all list elements
- Move the pointers around accordingly
- Careful with this

```
1      (APPEND '(1 2) '(3 4))           ; (1 2 3 4)
2      (APPEND '(1 2) '(3 4) '(5 6))    ; (1 2 3 4 5 6)
3      (APPEND '(A B C) 'D)              ; (A B C . D)
4      (APPEND '(A B C) (LIST 'D))       ; (A B C D)
```

APPEND Function

- Merges multiple lists into one
- Generally, expects all list elements
- Move the pointers around accordingly
- Careful with this

```
1      (APPEND '(1 2) '(3 4))           ; (1 2 3 4)
2      (APPEND '(1 2) '(3 4) '(5 6))    ; (1 2 3 4 5 6)
3      (APPEND '(A B C) 'D)              ; (A B C . D)
4      (APPEND '(A B C) (LIST 'D))       ; (A B C D)
```

APPEND Function

- Merges multiple lists into one
- Generally, expects all list elements
- Move the pointers around accordingly
- Careful with this

```
1      (APPEND '(1 2) '(3 4))           ; (1 2 3 4)
2      (APPEND '(1 2) '(3 4) '(5 6))    ; (1 2 3 4 5 6)
3      (APPEND '(A B C) 'D)              ; (A B C . D)
4      (APPEND '(A B C) (LIST 'D))       ; (A B C D)
```

APPEND Function

- Merges multiple lists into one
- Generally, expects all list elements
- Move the pointers around accordingly
- Careful with this

```
1      (APPEND '(1 2) '(3 4))           ; (1 2 3 4)
2      (APPEND '(1 2) '(3 4) '(5 6))    ; (1 2 3 4 5 6)
3      (APPEND '(A B C) 'D)             ; (A B C . D)
4      (APPEND '(A B C) (LIST 'D))      ; (A B C D)
```

APPEND Function

- Merges multiple lists into one
- Generally, expects all list elements
- Move the pointers around accordingly
- Careful with this

```
1      (APPEND '(1 2) '(3 4))           ; (1 2 3 4)
2      (APPEND '(1 2) '(3 4) '(5 6))    ; (1 2 3 4 5 6)
3      (APPEND '(A B C) 'D)              ; (A B C . D)
4      (APPEND '(A B C) (LIST 'D))       ; (A B C D)
```

Summary

- Lisp is built through S-Expression
- CONS for cell construction
- CAR and CDR to pull them apart
- LIST and APPEND for utility functions

Questions